Uma API ou classe **thread-safe** (segura para threads) em Java é aquela que pode ser usada por **múltiplas threads simultaneamente** sem causar corrupção de dados, inconsistências ou *race conditions* (condições de corrida). Já uma API **não thread-safe** (não segura) não possui esse controle interno; se várias threads tentarem ler e modificar o mesmo objeto ao mesmo tempo, o estado dele pode ser corrompido. [1, 2, 3, 4]

API Thread-Safe

- **O que é:** O próprio código da classe gerencia a concorrência (geralmente usando blocos `synchronized`, locks ou estruturas atômicas) para garantir que as operações ocorram de forma segura e previsível. [1, 2, 3]
- **Vantagens:** Não exige que você escreva código extra de sincronização ao compartilhar o objeto entre várias threads. [1]
- **Desvantagens:** Costuma ter um desempenho ligeiramente menor devido ao custo de travamento (*locking*) e sincronização.
- **Exemplos em Java:**
  - `ConcurrentHashMap` (em vez de `HashMap`)
  - `CopyOnWriteArrayList`
  - `Vector` e `Hashtable` (legados, mas thread-safe)
  - `StringBuffer` (em vez de `StringBuilder`)
  - Classes do pacote `java.util.concurrent.atomic` (ex: `AtomicInteger`, `AtomicLong`) [1]

API Não Thread-Safe (Non Thread-Safe)

- **O que é:** Uma classe ou método que **não** protege o acesso concorrente a seus dados internos. Se duas ou mais threads alterarem o mesmo objeto ao mesmo tempo, o resultado é indefinido. [1, 2, 3]
- **Vantagens:** Mais rápida e leve, pois não perde tempo com mecanismos de bloqueio. [1]
- **Desvantagens:** É responsabilidade do desenvolvedor implementar a sincronização manual (usando `synchronized`, `Locks`, etc.) se múltiplos fluxos forem acessar o recurso. [1, 2]
- **Exemplos em Java:**
  - `ArrayList`, `HashSet`, `HashMap`
  - `StringBuilder`
  - `SimpleDateFormat` (um clássico gerador de bugs concorrentes em aplicações web) [1, 2]

Resumo Comparativo

Característica	Thread-Safe	Não Thread-Safe
Acesso Concorrente	Seguro (gerencia sozinho)	Inseguro (pode corromper dados)
Desempenho	Menor (devido ao overhead de locks)	Maior (execução direta e rápida)
Uso Ideal	Estado compartilhado entre várias threads	Uso local em uma única thread ou com controle externo

Gostaria de ver um **exemplo prático em código** mostrando o que acontece quando uma `HashMap` (não thread-safe) é acessada por várias threads ao mesmo tempo, ou como usar uma `ConcurrentHashMap`?

- 

  DevMedia

  Thread-Safe Java: Entendendo o conceito e usando em sua aplicação

  O conceito de Thread-Safe é importante quando se trabalha com programação concorrente, ou seja, quando várias threads tentam acede...

- 

  DevMedia

  Artigo Java Magazine 69 - Aplicações concorrentes em Java

Mas desenvolver classes thread-safe exige muito mais atenção e análise do que classes não thread-safe.

Servlets containers, por ex...



- 



YouTube·Telusko

Thread Safety in Java

2 de jul. de 2019 — Thread safety is a computer programming concept applicable to multi-threaded code.

Thread-safe code only manipulates shared data s...



8m

Mostrar tudo

sim

Aqui está um exemplo prático. Vamos simular duas threads tentando adicionar elementos simultaneamente na mesma lista.

Primeiro, veremos o caos gerado pela **`ArrayList` (Não Thread-Safe)** e, logo depois, como a **`CopyOnWriteArrayList` (Thread-Safe)** resolve o problema sem mudar a lógica do seu código.

---

1. O Problema: `ArrayList` (Não Thread-Safe)

No código abaixo, criamos 2 threads. Cada uma tenta adicionar 1.000 números à mesma lista. O esperado seria uma lista final com exatamente 2.000 elementos.

java

```java
import java.util.ArrayList;
import java.util.List;

public class ExemploNaoSafe {
    public static void main(String[] args) throws InterruptedException {
        // ArrayList NÃO é segura para múltiplas threads
        List<Integer> lista = new ArrayList<>();

        // Thread 1: adiciona 1000 elementos
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                lista.add(i);
```

```
            }
        });

        // Thread 2: adiciona mais 1000 elementos
        Thread t2 = new Thread(() -> {
```

Use o código com cuidado.

O que acontece ao rodar esse código?

Como a `ArrayList` não tem travas internas, as duas threads tentam mexer no array interno e no contador de tamanho ao mesmo tempo. Isso causa três possíveis cenários (e você pode obter resultados diferentes toda vez que rodar):

1. **Tamanho incorreto:** O tamanho real pode dar `1985`, `1800`, etc., porque uma thread atropelou o incremento da outra.
2. **Erro em tempo de execução:** O programa pode quebrar e lançar uma `ArrayIndexOutOfBoundsException` no meio do caminho.

---

2. A Solução: `CopyOnWriteArrayList` (Thread-Safe)

Para corrigir isso de forma nativa e sem precisar colocar blocos de `synchronized` manuais, basta trocar a implementação para uma coleção do pacote `java.util.concurrent`.

java

```
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.List;

public class ExemploThreadSafe {
    public static void main(String[] args) throws InterruptedException {
        // CopyOnWriteArrayList É segura para múltiplas threads
        List<Integer> listaSafe = new CopyOnWriteArrayList<>();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                listaSafe.add(i);
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                listaSafe.add(i);
            }
```

Use o código com cuidado.

O que acontece aqui?

Não importa quantas vezes você rode este segundo código, o resultado será **sempre 2000**. A classe `CopyOnWriteArrayList` gerencia internamente os acessos para garantir que nenhuma modificação destrua os dados da outra thread.

Ficou clara a diferença na prática? Se quiser, posso explicar como funciona o mecanismo interno de travamento ou como aplicar a sincronização manual usando a palavra-chave `synchronized`!